

Histogram Optimization with CUDA

Keh Kok YONG
Accelerative Technology Lab
MIMOS Berhad
Kuala Lumpur, Malaysia
kk.yong@mimos.my

Sheera Shaheera OTHMAN TALIB
Accelerative Technology Lab
MIMOS Berhad
Kuala Lumpur, Malaysia
sheera.othman@mimos.my

Abstract— Histogram is a popular analytic graphical representation of data distribution resulting from processing a given numerical input data. Although the sequential histogram computation may be simple, it is no longer suitable in processing high volume of data. With recent advancement of high performance computing (HPC), aided by the accelerating growth of General Purpose Graphic Processing Unit (GPGPU), parallel implementation of this algorithm specifically for the NVIDIA architecture is explored. This paper presents various experimental analyses of parallel optimization of this algorithm. It is largely based on the advantages of utilizing multi-core CPU (OpenMP) and many-core GPU computing (Compute Unified Device Architecture, CUDA). The result shows the GPU optimized streaming histogram is able to gain 7x more speedup against the multi-threaded OpenMP implementation. Note that this processing time includes data transfer. It is important to have multiple rounds of fine tuning for different architectural platforms to increase parallelism.

Keywords—Histogram, GPU, CUDA, Parallel Processing, Optimization

I. INTRODUCTION

Histogram is a popular analytic graphical representation of data distribution resulting from processing a given numerical input data. It is one of the fundamental statistical mechanisms which are used in data analytics. Vast variety of application from different fields use histogram, such as computer vision, bioinformatics, image processing, pattern recognition, data mining, text analytics and others [1, 2, 3]. In the current era, the amount of data we deal with is growing tremendously with time.

The sequential histogram computation is simple, but it is no longer suitable to process high volume of data. Nonetheless, there are a wide number of parallel implementations available to better handle this. However, one of the key challenges for parallel processing is the irregularity of the memory access patterns. Multi worker threads might need to update the same buckets simultaneously. In this situation, a locking mechanism is essentially employed and as a result this kind of memory accesses will be serialized, causing a dip in performance. Yet, memory bandwidth and throughput remains as one of the crucial factors to gain better performance. Consecutively, these issues must be resolved for an efficient parallelization of the histogram computation.

Acceleration technology has been developed rapidly, alongside HPC technology with GPGPU. In 1980s, the first

generation of graphic processor, GE (geometry engine), made enormous impact on the development and revolution of computer graphics processing. In between mid-1980s and the end of 1990s, the direction of the mainstream technology has shifted from multi-core CPU to many-core GPU. Some of the major player in this industry are NVIDIA (Tesla), AMD (FirePro S-Series), Intel (Xeon Phi) and Altera. The hardware is designed for high compute density with throughput-optimized parallel architecture that further facilitated a steady performance growth.

Graphic Processing Unit (GPU) has already been used for general computing and not only for graphics purposes. It is capable to perform high volume and complex computations via the provided application programming interface (API). Additionally, it is one of the most energy efficient many-core hardware being utilized in supercomputing. The top 10 ranked energy efficient supercomputers¹ in the world such as the Sugon Cluster, Cray CS-Storm and Inspur TS10000 are using the NVIDIA Tesla GPUs alongside high speed network communication devices, such as Infiniband. Recently, new generations of powerful accelerators have also been announced. The Shoubu and Satsuki clusters use PEZY-SC accelerators with 1024 logic cores, while Sunway TaihuLight cluster adopts a many-core 64-bit RISC accelerator with 256 processing cores. These technologies provide computational scientists and researchers a way to address the world's most computationally demanding problems by several orders of compute efficiency.

In this paper, we present several experimental analyses of various approaches to parallel optimization for histogram algorithm. It is based on the advantages of utilizing multi-core CPU (OpenMP) and many-core GPU computing (CUDA). The study shows the effects of the optimization techniques employed at each level of implementation as well as the necessary tradeoffs. The remainder of this paper is structured as follows. Section II introduces previous studies on GPU-accelerated histogram computation. Section III describes the GPU programming model. Section IV details the optimization implementation methodology. Section V shows the experiment results obtained with several different optimizations. Finally, Section VI concludes the paper.

¹ The result was published in June 2016 by the Green500 energy-efficient supercomputers list. The ranking is taken from this website, <http://www.green500.org/?q=lists/green201606>

II. RELATED WORK

There are numerous studies on accelerating histogram computation using CUDA-compatible GPU. In 2007, Podlozhnyuk accelerated 256-bin histogram computation for image processing related applications [4]. This CUDA-based implementation uses the high speed on-chip shared memory to maximize the efficiency of histogram computations. Ugljesa explored the strategy where the computed data is made private to each thread to gain speed in performing sort-search with realistic inputs [5]. Lashgar and Baniyadi worked on using cache directives to specify variables which will utilize the shared memory for OpenACC² [6]. Ikeda, et al. implemented an optimizer that identifies histogram-related blocks in a naïve OpenACC code and automatically rewrites the identified blocks. As such multiple copies of histograms can be exploited for GPU acceleration [7].

There are various programming frameworks for accelerators with vendor specific language construct. Annotation-based approaches such as OpenMP [8] expect user to use the parallel directives alongside C/C++ language for multi-core CPU. While on the other hand, NVIDIA has developed its own proprietary framework known as CUDA. CUDA follows the C++ standard regarding function name mangling, while kernel names are machine generated and unknown to the developer. OpenCL [9] provides additional cross-platform parallel programming standard. Its core focus is on code portability across both CPUs and GPUs. However, OpenCL is not being used in our experiments as our works are mainly targeted on the NVIDIA GPU.

There are other histogram optimization techniques which focus on non GPU related architecture. Ranger et al. described a parallel implementation on MapReduce Framework, which utilized shared memory in task spawning and data communication [10]. It leads to scalable performance in both multi-core and symmetric multiprocessor. Korbinian worked on multi-core pattern-based parallelization runtime library which includes handling of parameter tuning [11]. Pankratius et al. attempted multi-threading implementation with speedup gain of 6x on an 8 core machine [12]. Gooitzen et al. used ZYNQ FPGA to accelerate HoG (Histogram of Gradient) that achieved 15x speedup and 5x reduction in latency over a quad core CPU [13]. Thus, there are various ways of implementing a better performing histogram computation with appropriate programming paradigm.

III. IMPLEMENTATION

Histogram is a function that counts the number of distinct elements from the input data. The calculated output shows the data distribution across predefined buckets. The set of input elements and the number of buckets can vary greatly depending on the application domain. A sequential design of histogram is as shown in Figure 1. An array “input” of size “n” stores the input data to be examined. These array elements are sampled to enumerate observed values. The array “histo” of size “buckets” represent a histogram. Its elements represent the

histogram’s buckets. Line 1 to 4 shows the initialization process of the “histo” array. Line 6 to 10 shows incremental value assigned to each bucket.

Parallel programming techniques can be used to accelerate histogram computation which can be divided into micro and macro parallelization levels. Micro level exploits data parallelism. It uses the same computational model for every thread, but feeds it with different parts of the data. The data to be examined is decomposed into smaller segments, which are then assigned to different threads. Each thread will then examine the assigned segment and update the histogram. It is also known as “Single Instruction Multiple Data” (SIMD). In general, it is well-suited for many-core accelerator. On the other hand, macro level works on simultaneous task execution, which is suitable for processing highly complex computations.

```

1  for (int i = 0; i < buckets; i++)
2  {
3      histo[i] = 0;
4  }
5
6  for (int i = 0; i < n; i++)
7  {
8      /* 0 <= input[i] < buckets */
9      histo[input[i]]++;
10 }
```

Figure 1: Simple Computation for Sequential Histogram

We implement the optimization of histogram computation by using both multi-core CPU and many-core GPU hardware. OpenMP multi-threaded directive method is adopted for the multi-core CPU implementation. To further exploit the many-core GPU compute capabilities and accelerator features, there are three different optimization mechanisms explored. It includes the use of global memory, shared memory and streaming. In this investigation, the input data is a random stream of 1 octet-wide, which may represent a stream of ASCII characters. As the 8-bits input can be anything ranging from 0x00 and 0xFF, a minimum of 2^8 number of buckets (bins) is required to keep count of the number of occurrences.

A. Multi-core CPU with OpenMP

The first attempt of histogram optimization is to use OpenMP to maximize the usage of multi-core CPU resources. It is a conventional optimization technique for multi-thread processing on a standalone machine. The parallel execution of the tasks is performed simultaneously across a defined number of CPU cores. However, in this implementation, due to the high possibility of multiple threads attempting to increase the same bin array element simultaneously, a local bin array per thread is necessary to avoid read-write conflicts. The final bin array is calculated by combining these local bin arrays when all threads have finished processing.

B. Many GPU Programming Model

This section provides a quick overview of the programming model for CUDA on NVIDIA GPU architecture. For the purpose of computation in GPU, data retrieved from a hard disk which is stored in the CPU memory needs to be transferred to GPU memory device via the PCIe bus. After GPU processing has been completed, the output will be transferred back to the CPU memory. These processes

² The OpenACC programming model has been developed to simplify accelerator programming and improved development productivity.

are initiated in the CPU host code. The core computation composed of functions and device API calls which are performed in the GPU, is referred to as kernel code. More detailed information can be found in [14].

CUDA uses the concept of threads, blocks, and grids. These kernel codes are executed in parallel by a series of threads. Blocks gather into thread-blocks to enable synchronization and the usage of shared memory, whereas grids group all the blocks. These worker threads are executed on streaming multiprocessors (SMXs). Thus, different blocks can be dynamically scheduled into different configurations such as concurrent block execution on different SMXs, concurrent execution on the same SMX by using CPU multi-threads; concurrent execution on the same or different multiprocessors at different times. All of the threads have dedicated registers but share the GPU global memory.

A warp is a group of multiple threads within a single block. It is a basic unit of thread scheduling in the SMXs. Once a block is assigned to a SMX, it is divided into units of warps. Each warp can support up to 32 threads in the Kepler architecture. Each thread in a warp executes in parallel. For optimized warp execution, branch divergence should be avoided as much as possible. Branch divergence occurs when threads inside a warp branch into different execution paths. All threads in the same grid execute the same kernel code but handle different data (SIMD). With the Kepler/Maxwell architecture, a block can consist of 1024 threads in each of the x, y, and z dimensions. The maximum of x dimension in blocks of grid can go up to 2^{32-1} .

There are three different GPU implementations: Global Memory, Shared Memory and Streaming. There are essential factors to be considered on the design of the kernel code in order to obtain the best performance such as data transfer latency, fine tuning of kernel parameters for data parallel execution and resource utilization. Both global and shared memory accesses should be performed in a coalesced manner. In such cases, GPU combines multiple load and store instructions from each thread within a warp into fewer numbers of transactions necessary to service all of the threads. As all memory accesses are cached, the number of transactions are dependent on the number of cache lines, 128 byte lines. Consequently, it keeps the workload efficiency high. Memory tiling and stride access techniques are employed to enhance the per thread workload. In the memory tiling implementation, each thread computes two adjacent input elements. In contrast, the stride access pattern implementation deliberately computes multiple non-adjacent data elements while striding across the input data.

1) Global Memory

Both the input data and bucket array are stored in the GPU global memory area. Each thread evaluates different elements of the input data buffer while increasing the bucket array elements accordingly. The same race condition similar to the CPU OpenMP implementation is highly probable, thus it is necessary to increment the bucket array atomically. An atomic operation is guaranteed to be performed with no interference from any other threads in the GPU. Consequently, this atomic operation might cause some of the write operations to be

serialized instead, which in turn may dampen performance. Figure 2 shows the globally allocated input and bucket memory locations are supplied via the kernel's function parameters. The "cudaMalloc()" CUDA API is called to allocate the GPU memory in the main caller section code before launching the kernel.

```

1  __global__ void
2  gpu_histogram_stride_access(
3      unsigned char * data,
4      unsigned int * bin,
5      unsigned long long size)
6  {
7      unsigned long long int i;
8      unsigned int offset;
9      i = blockIdx.x * blockDim.x + threadIdx.x;
10     offset = blockDim.x * gridDim.x;
11     while (i < size) {
12         atomicAdd(&bin[data[i]], 1);
13         i += offset;
14     }
15 }

```

Figure 2: Global Memory Stride Access Kernel Code

2) Shared Memory

This optimization uses the GPU shared memory where its latency is roughly 100x lower than global memory latency with appropriate coalesced access. Each thread has its own private copy of the bucket array. Input data is stored in the global memory, while the private bucket array is located in the shared memory, as shown in Figure 3, where the private copy is declared using the `__shared__` variable specifier. Each thread utilizes stride access across the assigned input data, and then increases these local buckets. Therefore, it does not directly access the final bucket array in the global memory. After all threads have finished processing, ensured by the synchronization barrier, the final bucket array is updated with the sum of all of these local buckets.

```

1  __global__ void
2  gpu_histogram_shared_mem_stride_access(
3      unsigned char * data,
4      unsigned int * bin,
5      unsigned long long size)
6  {
7      unsigned long long i;
8      unsigned int stride;
9      __shared__ unsigned int shared_bin[BIN_SIZE];
10     shared_bin[threadIdx.x] = 0;
11     __syncthreads();
12     i = blockIdx.x * blockDim.x + threadIdx.x;
13     stride = blockDim.x * gridDim.x;
14     while (i < size) {
15         atomicAdd(&shared_bin[data[i]], 1);
16         i += stride;
17     }
18     __syncthreads();
19     atomicAdd(&bin[threadIdx.x],
20             shared_bin[threadIdx.x]);
21 }

```

Figure 3: Shared Memory with Stride Access Kernel Code

3) Streaming

In the final optimization phase, the kernel implementation with the best performance is modified to perform computation in multiple streams. In essence, the compute cycle of one kernel overlaps with the host to device data transfer of another kernel, thus improving utilization efficiency. In order to stream, one of the requirements is to use host pinned memory

for the input data using the API “cudaMallocHost()”. This pinned memory is generally used as a staging area when transferring data from host pageable memory to the device. This will allow the CUDA driver to directly copy these data into the device.

```

1  for (int j = 0; j < no_streams; j++)
2  {
3      checkCuda(cudaMemcpyAsync(d_data +
4          (j * stream_size * sizeof(unsigned char)),
5          h_data + (j * stream_size *
6              sizeof(unsigned char)),
7          stream_size * sizeof(unsigned char),
8          cudaMemcpyHostToDevice, streams[j]));
9
10     gpu_histogram_shared_mem_stride_access
11     <<<dimGrid, dimBlock, 0, streams[j]>>>
12     (d_data + (j * stream_size *
13         sizeof(unsigned char)), d_bin,
14         stream_size);
15 }
16
17 cudaDeviceSynchronize();

```

Figure 4: Code Snippet of Stream Implementation

As shown in Figure 4, the same kernel is launched in all of the streams created for data processing except for the default stream. No kernel should be launched on the default stream to avoid serialization. The API “cudaMemcpyAsync()”, is used for asynchronous memory transfer from the pinned host memory to the device on the non-default streams. By doing this, data transfer and kernel processing between different streams are overlapped.

IV. EVALUATION & RESULT

This section describes the setup of the test environment, shows the experiment results and discusses various implemented optimization mechanisms for the parallelization of histogram.

Table 1: Specification of K20c and GTX980Ti

Card	K20c	GTX980Ti
CUDA Cores	2496	2816
Memory Bus	320-bit	384-bit
Memory Size	5GB GDDR5	6GB GDDR5
Memory Bandwidth	208 GB/s	336 GB/s

A. Test Environment

The experiments are executed on a workstation, which contains Intel Xeon Processor E5-1607@3GHz, 8GB DDR3 RAM and 400GB SATA hard disk storage. The operating system is Ubuntu 14.04 TLS (x64) with NVIDIA Tesla driver 352.93. There are two different types of NVIDIA GPU cards used for testing; NVIDIA Kepler K20c and GeForce Maxwell GTX980Ti. The specification is shown in Table 1. The source codes are compiled using NVIDIA NVCC. It is based on CUDA 7.5 toolkit with compute capabilities version 3.5 for NVIDIA Kepler architecture. The Nsight Eclipse IDE tool is used for code editing and performance profiling.

B. Methodology

The size of input data ranges from 512MB to 3584MB. The measurement for each individual kernel code speedup is

derived according to Equation 1, while using the sequential CPU implementation performance as a base.

$$Speedup = \frac{CPU\ Computation\ Time}{GPU\ Computation\ Time}$$

Equation 1: The speedup calculation.

Final average speedup is calculated by averaging the individual speedup achieved for each of the kernel codes using the varying input data size. These measurements are taken for multiple runs in order to reduce uncertainties and increase accuracy.

C. Results

Figure 5 shows the average speedup of each kernel implementation from all of the optimization phases.

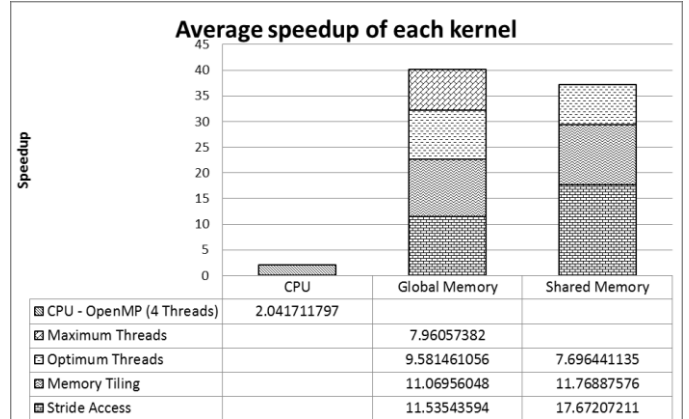


Figure 5: Graph of the average speedup for both global and shared memory

The average speedup of the GPU implementation in the first optimization phase using global memory with maximum threads per block shows almost 8x speedup comparing to the CPU OpenMP implementation. Although this implementation has utilized all available CPU cores from the workstation, the sheer number of threads executed on the GPU is simply enormous. The GPU architecture is designed to be massively parallel with its thousands of much smaller CUDA cores. As an example, the NVIDIA Tesla K20 consists of 2496 cores allowing the GPU to efficiently handle multiple tasks simultaneously.

In the same global memory implementation, one of the kernel parameters is then tuned to allot 256 threads per block. The performance of this particular kernel gains 3x more speedup. Thus, this is deemed as an optimum configured block-thread parameter. Although this is a successful optimization, the Nsight profiler has shown it has a lower occupancy rate compared to the previous kernel as shown in Figure 6. This leads to the realization that optimum parameters may not necessarily mean maximum usage of the GPU resources.

Although the occupancy has slightly reduced, the number of atomic request transactions presented in Figure 7 has equally reduced by at least 76 million. This reduction is a result of aligning the number of threads per block to the number of bucket array where the possibility of read-write access conflicts on the same element has declined. This seems

to suggest that the reduced number of atomic request transactions has contributed to the slight increase in performance.

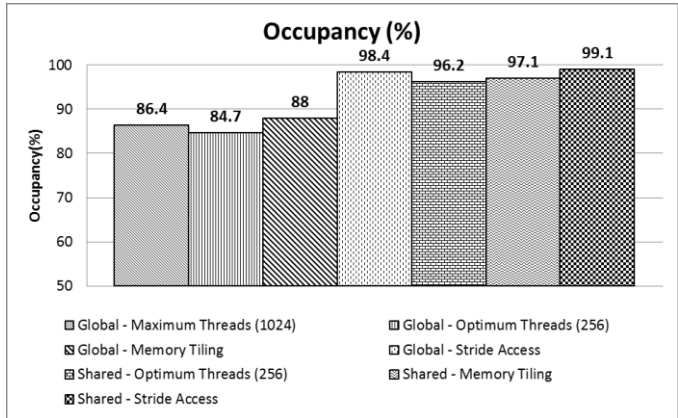


Figure 6: GPU Occupancy from NVIDIA Nsight

While the global memory tiling implementation managed to further boost the speedup to 11x by increasing the occupancy by slightly less than 5% compared to the global optimum threads implementation, the atomic request transactions has surged up by 3 million. Moreover, the memory tiling implementation has caused the memory access to be performed in a non-coalescing manner as shown in Figure 7. This appears to suggest that the instruction level parallelism introduced within this kernel is the contributing factor in the performance enhancement.

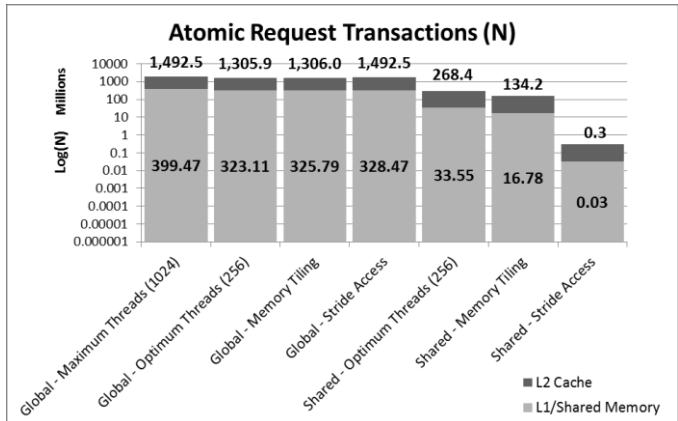


Figure 7: Atomic requests transactions

With the global stride kernel implementation where the instruction level parallelism increases by adding more computation load per thread, the occupancy has risen up to slightly more than 98% while keeping the global load to be 100% efficient. Yet, the speedup growth is only minimal despite the fact that the atomic transactions were able to be reduced by almost 200 million. In the second GPU optimization phase, the usage of the shared memory has shown that it does not guarantee better performance. The speedup is comparable or slightly lower than the first GPU implementation (Global Memory) by maximizing the threads usage. Other than that, this kernel implementation has tremendously reduced the less favorable usage of atomic

transactions to a mere total of 300 million transactions, a reduction of almost 1.5 billion.

Table 2: The GPU kernel implementations global load efficiency.

Functions		Global Load Efficiency (%)
Global Memory	Maximum Threads	100
	Optimum Threads	100
	Memory Tiling	50
	Stride Access	100
Shared Memory	Optimum Threads	100
	Memory Tiling	50
	Stride Access	100

However, in the shared memory stride access kernel implementation, the speedup has shot up to more than 17x compared to the sequential implementation. This kernel has achieved the highest occupancy of 99% and the lowest number of atomic transactions at less than 300 thousand transactions. This seems to suggest the importance of the atomic request transactions to the performance of the kernel.

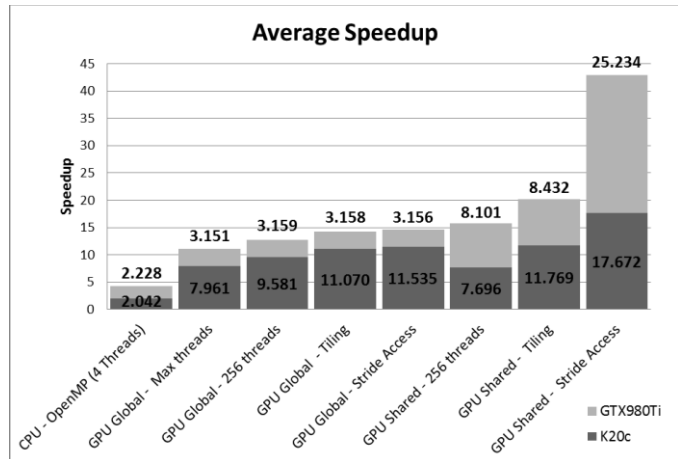


Figure 8: Average speedup of all kernels on both GPU cards.

Figure 8 shows the comparison of executing the kernels on different GPU architectures. Due to the fact that these kernels are optimized on the K20c GPU card, these kernels are performing poorly on the GTX980Ti GPU card. However, a massive growth of an order in speedup is displayed by the GPU shared memory stride access implementation in comparison. The Maxwell architecture has a faster shared memory atomic operation which may have contributed to this increment. The atomic operation is implemented using a native shared memory 32/64 bit compare-and-swap (CAS) instead of the lock-update-unlock implementation which can be found on both Fermi and Kepler architecture.

Figure 9 shows the comparison of the overall processing time inclusive of memory transfer for the shared memory stride access kernel for both streaming and non-streaming implementation. As shown here, the overall speedup of the non-streaming implementation inclusive of the memory transfer is dramatically reduced to merely 6x faster, even using the most optimized kernel available that has an almost 18x speedup. This is due to limitations on the memory transfer speed which is the key contributing factor to this reduction. Hence, it is important to implement optimization techniques

such as streaming which could possibly help in diminishing this memory transfer latency and improve the overall performance.

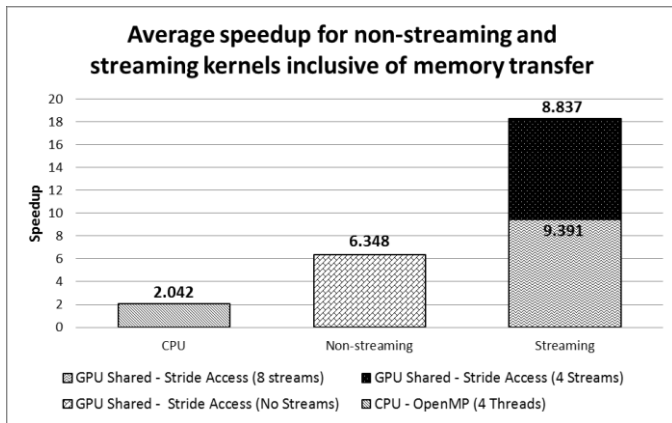


Figure 9: Average speedup of CPU & GPU non-streaming and streaming

By streaming the kernel into 4 streams, a further speedup of almost 1.5 times can be gained. The time profile of this computation is also displayed in Figure 10. As shown here, the data transfer from host to device and the kernel computation are overlapped. Hence, some of the memory latency during the data transfer is effectively hidden and as a result the performance is immediately enhanced. However, the average speedup attained by doubling the number of streams is only minimal, which seems to suggest that there's a limit to the effective number of streams before the speedup saturates.

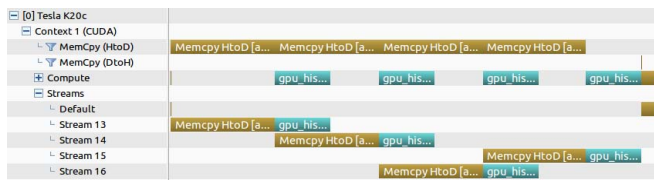


Figure 10: Time profile of the streaming implementation (4 streams)

Nevertheless, from Figure 11, with increasing number of streams, a significant growth can be achieved using the GTX980Ti GPU card. Streaming on the Maxwell GPU card allows the application to achieve an additional 2 to 5 times more speedup. In conclusion, with the adoption of streaming, a maximum of 7x and 12x speedup is achieved respectively for both K20c and GTX980Ti GPU card compared to the CPU OpenMP implementation.

V. CONCLUSION

In this work, we presented and evaluated various optimizations on multi-core CPU and many-core GPU for the histogram algorithm. The optimization of an algorithm on the GPU architecture has different challenges compared to the CPU. We observed that the best GPU optimization uses CUDA streams. In addition, a higher amount of shared memory available can certainly boost up performance.

Furthermore, there is still performance bottleneck of PCIE bus while transferring data between host and device. In the future, work on tuning mechanism that automates configuring the optimum parameters for various GPU devices can be

explored. One approach to this solution may be to tune the scheduler such that better stream overlapping can be achieved. Other than that, an execution plan can be developed which will avoid stalls due to data dependencies in extensive cache usage scenarios, improve branch prediction and prevent divergence. Expectantly, these steps would further enhance the performance of the application.

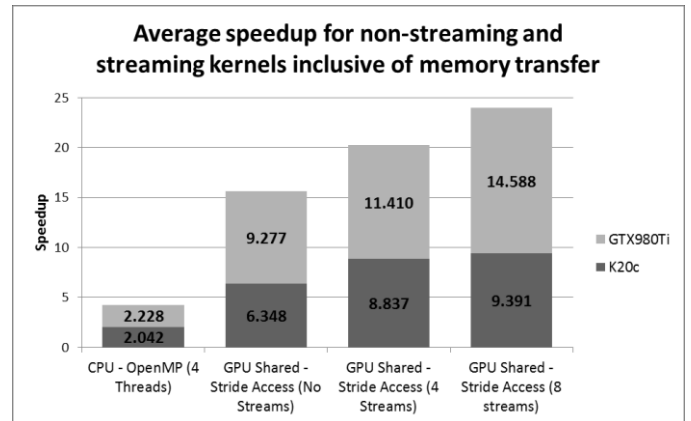


Figure 11: Average speedup of K20C and GTX 980Ti

VI. REFERENCES

- [1] S. Calarasanu, J. Fabrizio, and S. Dubuisson, "Using histogram representation and Earth Mover's Distance as an evaluation tool for text detection," in *Document Analysis and Recognition (ICDAR), 2015 13th International Conference on*, 2015, pp. 221–225.
- [2] D. Shyam, Y. Wang, and A. C. Kot, "Histogram-Based Fast Text Paragraph Image Detection," in *Computational Intelligence, 2015 IEEE Symposium Series on*, 2015, pp. 473–480.
- [3] C. Vetter and R. Westermann, "Optimized GPU histograms for multi-modal registration," in *2011 IEEE International Symposium on Biomedical Imaging: From Nano to Macro*, 2011, pp. 1227–1230.
- [4] V. Podlozhnyuk, "Histogram calculation in cuda," 2007.
- [5] U. Milic, I. Gelado, N. Puzovic, A. Ramirez, and M. Tomasevic, "Parallelizing general histogram application for CUDA architectures," in *Embedded Computer Systems: Architectures, Modeling, and Simulation (SAMOS XIII), 2013 International Conference on*, 2013, pp. 11–18.
- [6] A. Lashgar and A. Baniasadi, "Employing Software-Managed Caches in OpenACC: Opportunities and Benefits," *ACM Trans. Model. Perform. Eval. Comput. Syst.*, vol. 1, no. 1, pp. 2:1–2:34, Feb. 2016.
- [7] K. Ikeda, F. Ino, and K. Hagihara, "An OpenACC Optimizer for Accelerating Histogram Computation on a GPU," in *2016 24th Euromicro International Conference on Parallel, Distributed, and Network-Based Processing (PDP)*, 2016, pp. 468–477.
- [8] R. Vidal, M. Casas, M. Moretó, D. Chasapis, R. Ferrer, X. Martorell, E. Ayguadé, J. Labarta, and M. Valero, "Evaluating the Impact of OpenMP 4.0 Extensions on Relevant Parallel Workloads," in *OpenMP: Heterogeneous Execution and Data Movements: 11th International Workshop on OpenMP, IWOMP 2015, Aachen, Germany, October 1-2, 2015, Proceedings*, C. Terboven, R. B. de Supinski, P. Reble, M. B. Chapman, and S. M. Müller, Eds. Cham: Springer International Publishing, 2015, pp. 60–72.
- [9] D. R. Kaeli, P. Mistry, D. Schaa, and D. P. Zhang, *Heterogeneous Computing with OpenCL 2.0*, 3rd ed. Morgan Kaufmann, 2015.
- [10] C. Ranger, R. Raghuraman, A. Penmetsa, G. Bradski, and C. Kozyrakis, "Evaluating MapReduce for Multi-core and Multiprocessor Systems," in *2007 IEEE 13th International Symposium on High Performance Computer Architecture*, 2007, pp. 13–24.
- [11] K. Molitorisz, T. Müller, and W. F. Tichy, "Patty: A Pattern-based Parallelization Tool for the Multicore Age," in *Proceedings of the Sixth International Workshop on Programming Models and Applications for*

- Multicores and Manycores*, 2015, pp. 153–163.
- [12] V. Pankratiy, C. Schaefer, A. Jannesari, and W. F. Tichy, “Software Engineering for Multicore Systems: An Experience Report,” in *Proceedings of the 1st International Workshop on Multicore Software Engineering*, 2008, pp. 53–60.
 - [13] G. van der Wal, D. Zhang, I. Kandaswamy, J. Marakowitz, K. Kaighn, J. Zhang, and S. Chai, “FPGA acceleration for feature based processing applications,” in *2015 IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2015, pp. 42–47.
 - [14] D. Storti and M. Yurtoglu, *CUDA for Engineers: An Introduction to High-Performance Parallel Computing*, 1st Editio. Addison-Wesley Professional, 2015.